# Fun with Flow Matching!

From restoring noisy photos to making visual illusions, this project explores what a learned velocity field can do. First I use a pretrained image model, then train small UNets on handwritten digits.

Full report PDF · Part A PDF · Part B PDF

## Part A / Exploring pretrained flow models

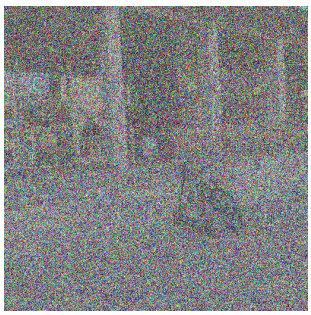
I use PixNerd-XXL-P16-T2I to predict a velocity field for 512 × 512 RGB images. All integration states and model evaluations use float32. Time runs from noise at 0 to a clean image at 1. The building and matcha photos are mine, reused from Project 2. The flower and house are my hand-drawn sketches.

$x_t = (1 - t)\varepsilon + tx_1$; target velocity = $x_1 - \varepsilon$

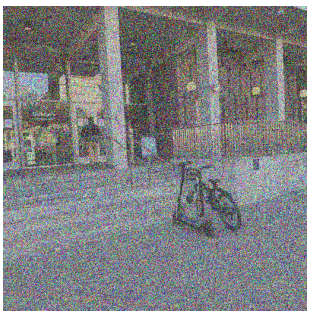
### 1.1 Constructing noisy inputs

I center-crop the building photo to a square, resize to 512 × 512, and map RGB to [−1, 1]. The same Gaussian noise tensor, seed 180, is reused at all three times. Increasing t retains more of the photo. The endpoint checks return exactly the noise at t = 0 and the original at t = 1.

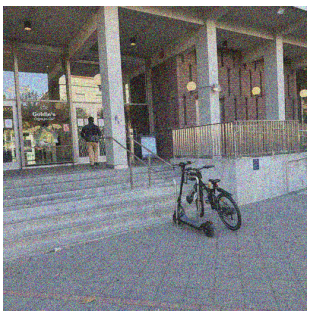
original



t=0.25



t=0.5



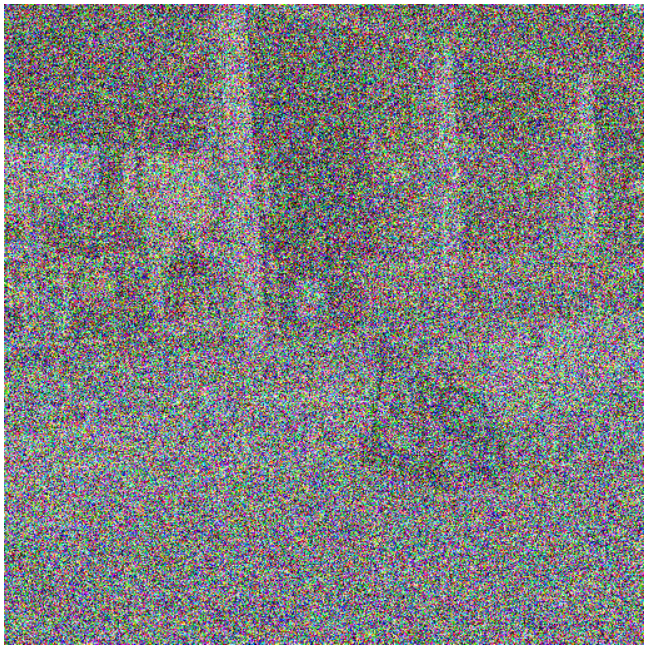
t=0.75

### 1.2 Gaussian denoising

I divide each noisy input by t before blurring to restore the image amplitude. I compare $\sigma \in \{1, 2, 4, 8, 16\}$ and choose the lowest reconstruction MSE against the known original. This is an image-specific baseline: t = 0.25: σ = 8, t = 0.5: σ = 4, t = 0.75: σ = 1.

At t = 0.25, the large blur needed to suppress noise also removes the railing, bike, and paving detail. At t = 0.50 the building layout is clearer, but colored noise remains. At t = 0.75, much less blur is needed because the original contributes more strongly to the input.

t = 0.25



noisy

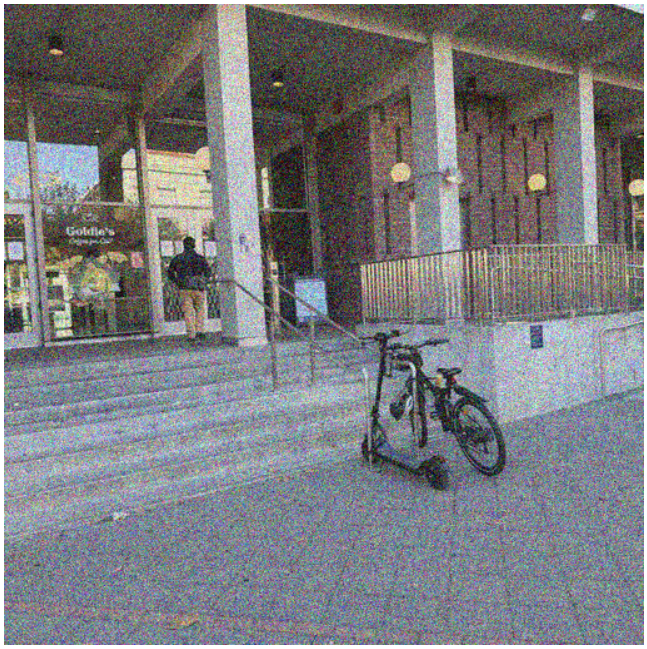
blur sigma=8.0

t = 0.50

noisy

blur sigma=4.0

t = 0.75



noisy

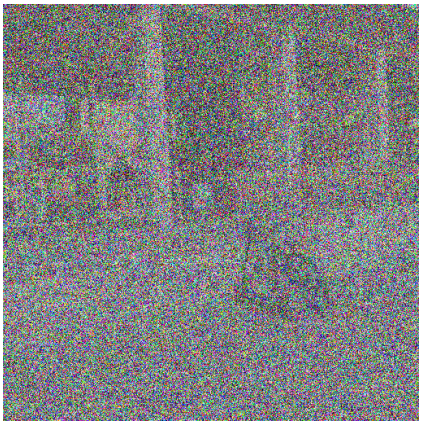
blur sigma=1.0

# 1.3 One-step denoising

With the prompt “a high quality photo”, one velocity prediction estimates the endpoint as $\hat{x}_1 = x_t + (1 - t)\hat{v}$. The multiplier is the remaining time. Even at t = 0.25, the model recovers the broad building layout and the bike position, but edges and textures are soft. More of the original detail survives at later times. A single prediction cannot fully account for how the learned velocity changes along the remaining path.

t = 0.25

original



noisy

one step

t = 0.50

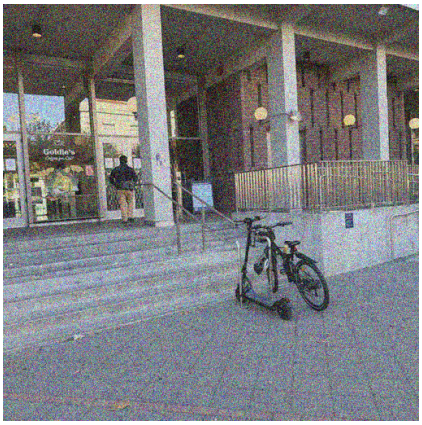
original

noisy

one step

t = 0.75

original



noisy

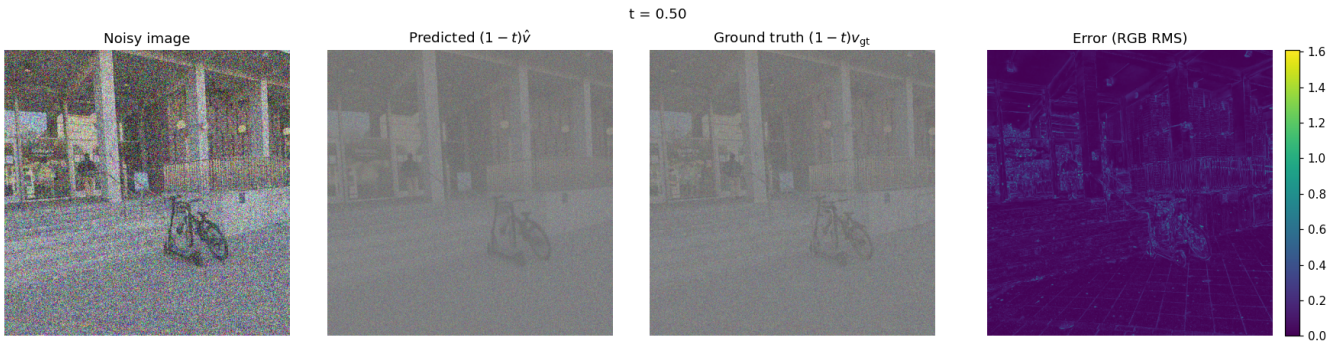
one step

# 1.4 What the model adds

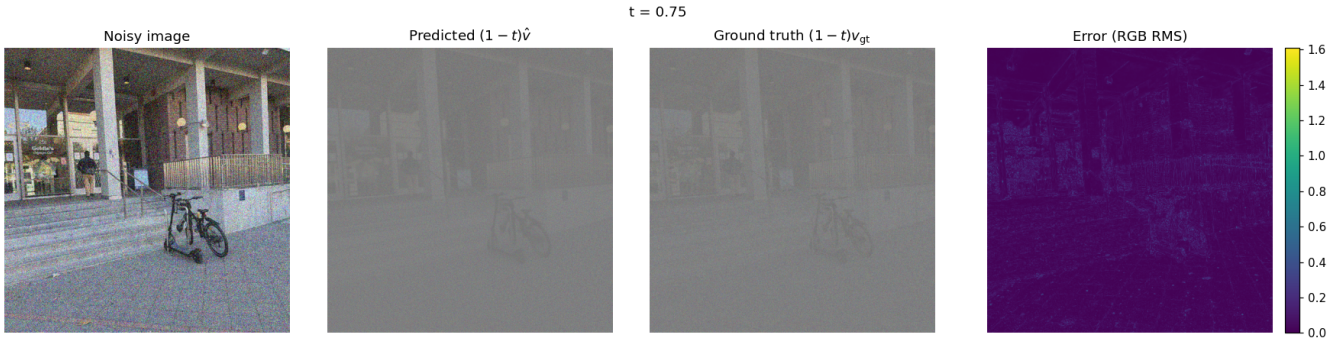
The predicted update is $(1 - t)\hat{v}$; the exact update is $x_1 - x_t$. For each pixel, I square the RGB update differences, average across the three channels, then take the square root. These errors are computed before clipping. All signed updates use the same symmetric range ±4.336, with zero at middle gray. All heatmaps share the linear range [0.000, 1.612].

Original building photo

t = 0.25: noisy input, predicted update, exact update, RGB RMS error




t = 0.50: noisy input, predicted update, exact update, RGB RMS error



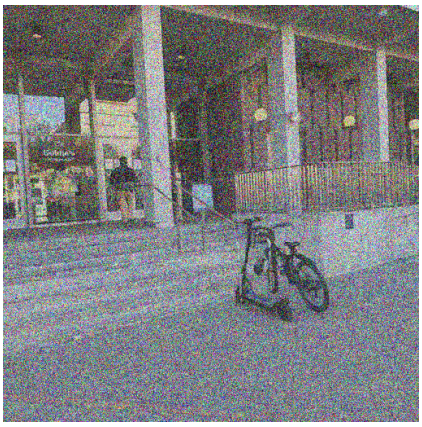

t = 0.75: noisy input, predicted update, exact update, RGB RMS error

The predicted and exact updates have similar noise-cancelling structure, but the error map highlights fine edges such as the railing, bike, and paving joints. These are details that the one-step estimate smooths or changes. The shared display limits make the three times comparable.

# 1.5 Euler denoising

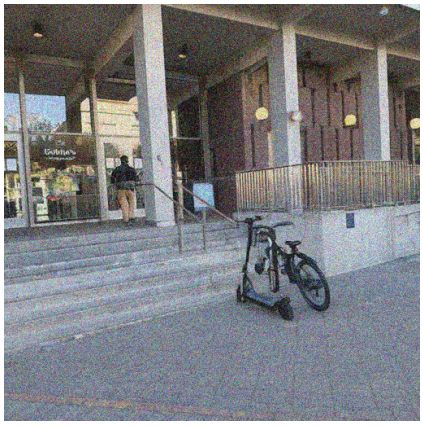
I recompute velocity after every update $x \leftarrow x + hv(x, t)$, using $h = 0.02$. From $t = 0.50$, this takes 25 updates. The trajectory includes the initial state, every fifth update, and the endpoint. Constant-velocity checks pass for 50 steps, 25 steps, and a shortened final step starting at 0.93.

t=0.50

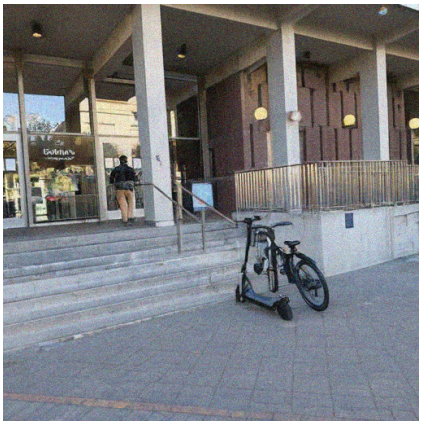


t=0.60

t=0.70



t=0.80



t=0.90

t=1.00

original

blur

one step

Euler

| Method at t = 0.50 | Reconstruction MSE in [−1, 1] |
|---|---|
| Gaussian blur | 0.035854 |
| One step | 0.009686 |
| Euler | 0.011088 |

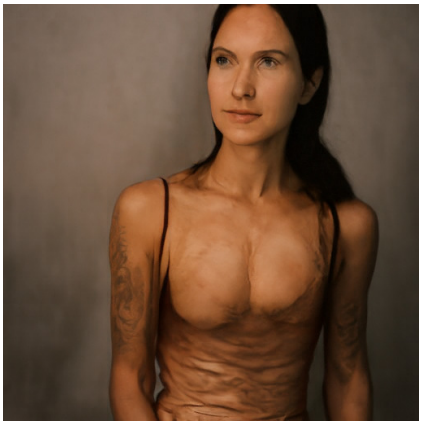
Euler produces sharper paving, brickwork, and bike details than the one-step estimate. However, it also changes small structures instead of exactly restoring them. Here one-step denoising has the lowest pixel MSE, while Euler looks sharper. This illustrates that plausible learned detail and pixel-accurate reconstruction are different goals.

# 1.6 Generating from pure noise

Each image starts from independent Gaussian noise and uses 50 Euler updates with the prompt “a high quality photo”. The seeds are 180–184.

All five results tend toward portraits despite the generic prompt. Some have malformed facial details, especially the first and fourth samples. This small set suggests a strong portrait tendency for this prompt and seed range; it is not a complete characterization of the training data.

seed=180



seed=181

seed=182

seed=183

seed=184

## 2.1 Classifier-free guidance

$v_{cfg} = v_c + w(v_c - v_u)$, $w = 7$

The unconditional branch uses the empty prompt. Both predictions receive the same state and time. I verify that $w = 0$ equals the conditional prediction exactly. These images reuse the five initial noise tensors from Section 1.6.

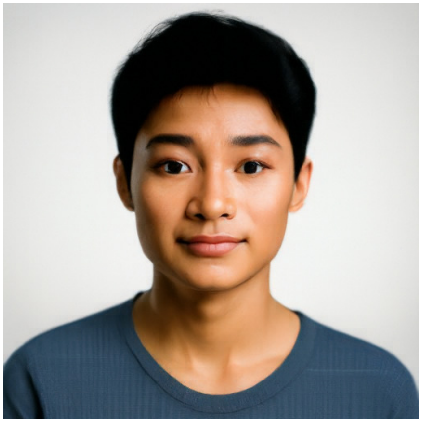
Guidance makes faces cleaner and more sharply defined for these same five seeds, reducing several obvious facial artifacts. It also changes identities, clothing, and composition, rather than simply sharpening the earlier image. The results remain concentrated on portraits and have a more uniform portrait style.

seed=180, w=7

seed=181, w=7

seed=182, w=7

seed=183, w=7

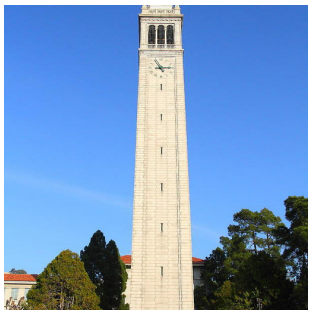


seed=184, w=7

# 3.1 Image-to-image editing

I interpolate each source with the same seed-180 noise at seven starting times, then integrate to t = 1. All edits use h = 0.02, w = 7, an empty unconditional prompt, and “a high quality photo” as the conditional prompt. Smaller starting times leave less evidence of the source and allow larger changes.

## Campanile



original

t=0.02

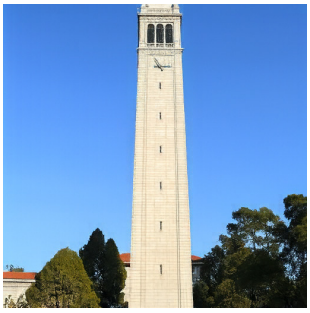
t=0.04

t=0.06

t=0.10

t=0.14

t=0.25



t=0.50

## My building photo

original

t=0.02

t=0.04

t=0.06

t=0.10

t=0.14

t=0.25

t=0.50

## My matcha photo

original

t=0.02

t=0.04

t=0.06

t=0.10

t=0.14

t=0.25

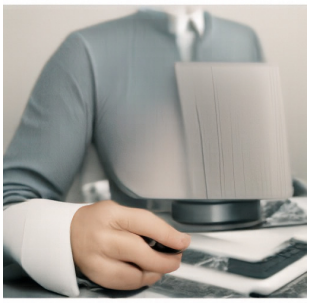
t=0.50

## Web image: Hokusai’s Great Wave

original

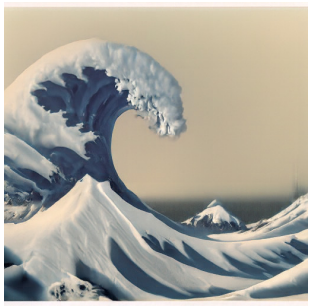
t=0.02

t=0.04



t=0.06

t=0.10



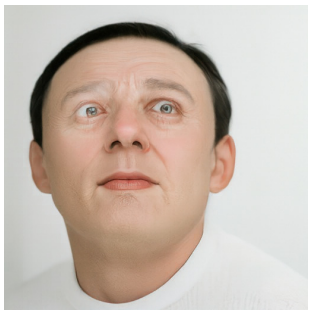
t=0.14

t=0.25

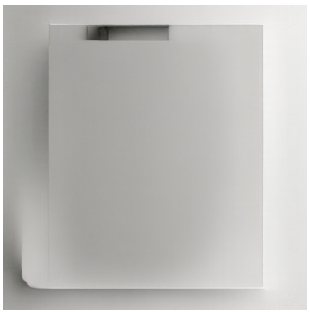
t=0.50

## My flower sketch

original

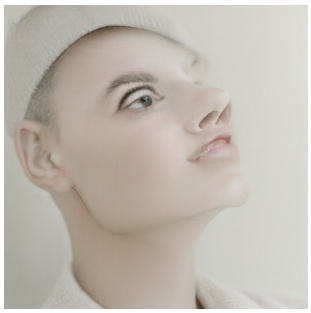


t=0.02



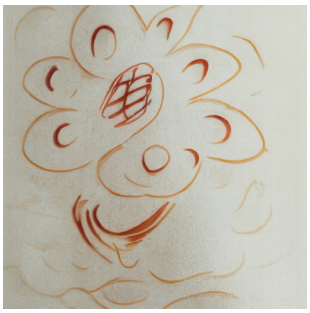
t=0.04

t=0.06



t=0.10

t=0.14



t=0.25

t=0.50

My house sketch

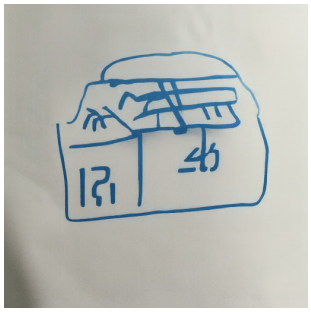
original

t=0.02

t=0.04

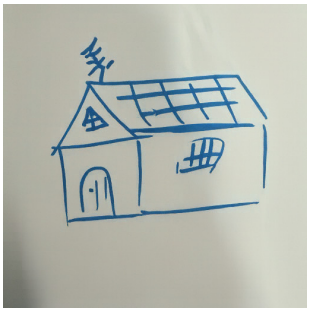
t=0.06

t=0.10



t=0.14

t=0.25



t=0.50

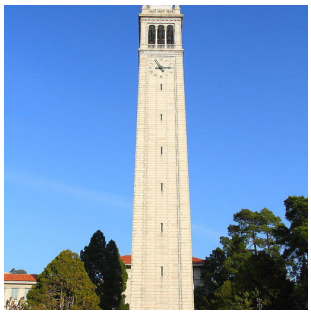
At t = 0.50, all inputs remain close to their sources. At smaller times, the generic prompt can overwhelm the source: the building and matcha sometimes become portraits or unrelated objects. The web print becomes a more photographic-looking wave around t = 0.10–0.14. The two sketches mostly remain drawings at t = 0.25–0.50; earlier times often produce unrelated scenes rather than real flowers or houses. The generic prompt does not reliably preserve the intended subject of a sketch.

# 3.2 Text-guided editing

I reuse the Campanile and both personal photos, all seven starting times, the same noise, and the same guidance settings. Only the text prompt changes.

## Campanile → rocket

Prompt: “a photograph of a rocket on a launch pad”

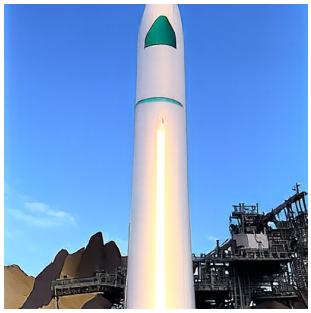


original

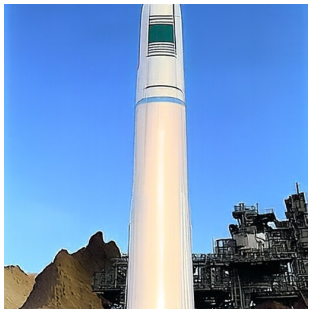
t=0.02

t=0.04

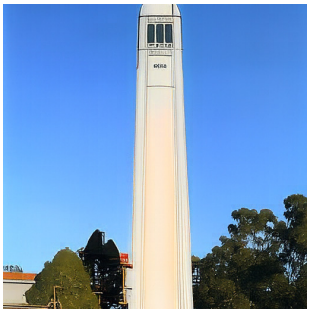
t=0.06



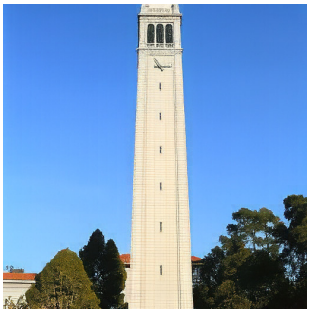
t=0.10



t=0.14



t=0.25



t=0.50

## Building → castle

Prompt: “a photograph of a medieval stone castle entrance”

original

t=0.02

t=0.04

t=0.06

t=0.10

t=0.14

t=0.25

t=0.50

### Matcha → milkshake

Prompt: “a photograph of a strawberry milkshake with whipped cream”

original

t=0.02

t=0.04

t=0.06

t=0.10

t=0.14

t=0.25

t=0.50

The prompts give more useful control at intermediate times. The building becomes a stone entrance or courtyard around t = 0.04–0.14; at t = 0.25, parts of the original pillars and bike remain while stone textures appear. The matcha becomes a red drink with strawberries and whipped cream at early and intermediate times. At t = 0.50 the original green drink dominates. The Campanile similarly trades source structure for the rocket prompt as the starting time decreases.

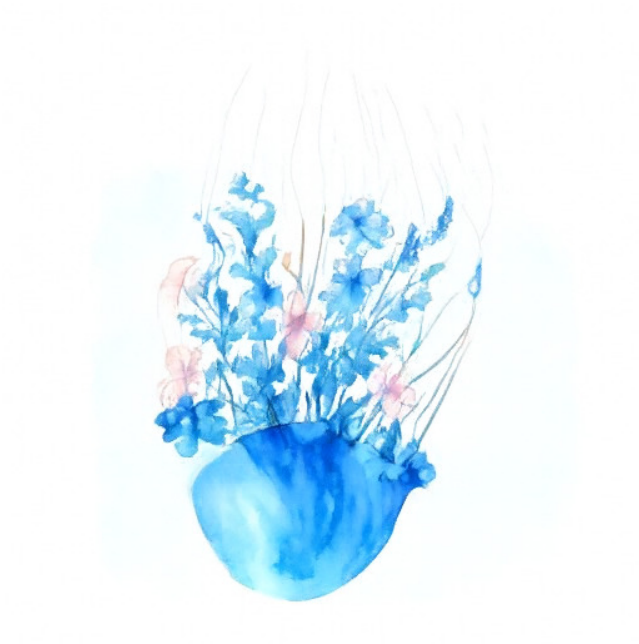
## 3.3 Visual anagrams

At each Euler step, I average the guided velocity in the normal orientation with the guided velocity of the 180°-rotated image, rotated back. Rotation flips both spatial axes. Applying it twice restores the original tensor exactly. Both experiments use h = 0.02 and w = 7.

## Jellyfish / bouquet of flowers

Seed: 180.

a watercolor of a jellyfish underwater



a watercolor of a bouquet of flowers

## Mushroom / hot-air balloon

Seed: 181.

a storybook illustration of a mushroom

a storybook illustration of a hot-air balloon

The jellyfish bell reads as a blue vase after rotation, while the tentacles become flower stems. Blue and pink petals are visible in both orientations, so the two readings share recognizable details. The mushroom/balloon pair is less clean: upright, an orange cap and pale stem-like shapes suggest a mushroom; rotated, the rounded lower region becomes the balloon envelope and the small shape below becomes a basket. The doubled stem and visible basket structure weaken the mushroom reading. I compared five mushroom/balloon candidates and retained the original seed-181 result; extra seeds and more detailed prompts did not improve both readings together.

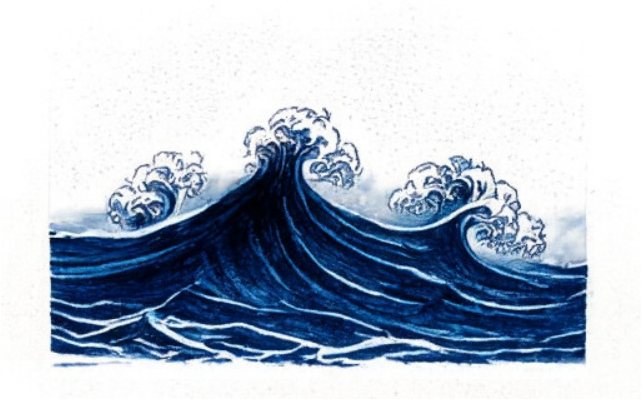
# 3.4 Hybrid images

$v = \text{lowpass}(v_a) + v_b - \text{lowpass}(v_b)$

The low-frequency prompt supplies the distant structure; the high-frequency prompt supplies the close detail. I use a 265 × 265 Gaussian kernel with σ = 16 at every step, h = 0.02, and w = 7. The small previews below simulate viewing from farther away.

## Mountains / ocean waves

Far: “a lithograph of three tall mountain peaks silhouetted against a white sky”. Close: “a lithograph of ocean waves”. Seed: 42.



Close view: ocean waves

Distant view: mountain range

## Whale / galaxy

Far: “an oil painting of a dark whale silhouette in side view on a pale background”. Close: “an oil painting of a galaxy”. Seed: 43.

Close view: galaxy

Distant view: whale

The mountain/wave hybrid uses a broad triangular dark ridge as mountain-like structure at a distance and curling crests as ocean detail up close. The waves remain recognizable in the small preview, so the mountain interpretation is weak rather than a clean perceptual switch. In the whale/galaxy hybrid, the large dark mass and tail-like extension suggest a whale from farther away, while close inspection reveals stars and diffuse galaxy texture. Its contour and flippers are indistinct, so the animal reading remains ambiguous. For each pair I compared five candidates, including the original short prompt and two more explicit compositions with two seeds each. The selected seeds are 42 and 43. These were more readable than the initial trials, but neither fully separates its two subjects.

# Part B / Training flow matching from scratch

I train on all 60,000 MNIST training images, scaled to [0, 1]. The 10,000 test images are reserved for visual evaluation and are never used for optimizer updates. Each batch gets fresh independent Gaussian noise. All runs use seed 180, Adam, and float32 tensors. The denoiser ran on a Tesla T4; the remaining runs used an A100. Losses below are mean training MSE, not test error or a measure of image quality.

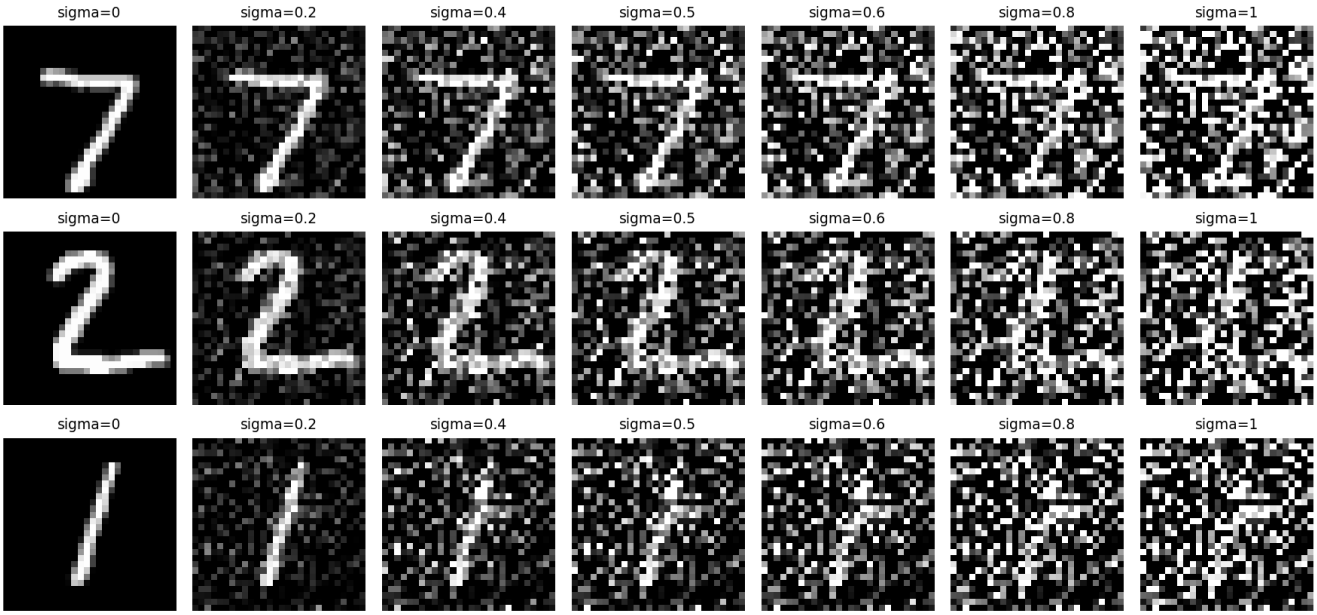
## 1.1 UNet architecture

Each Conv block is a 3 × 3 convolution, batch normalization, and GELU. ConvBlock applies it twice. DownConv uses stride 2; UpConv is a 4 × 4 transposed convolution with stride 2. The bottleneck averages over the 7 × 7 map, then unflattens with a 7 × 7 transposed convolution. Skip connections concatenate encoder features at matching resolutions.

| Stage | Feature shape, excluding batch |
|---|---|
| Input → ConvBlock | 1 × 28 × 28 → D × 28 × 28 |
| Down 1 → Down 2 | D × 14 × 14 → 2D × 7 × 7 |
| Flatten → Unflatten | 2D × 1 × 1 → 2D × 7 × 7 |
| Concat bottleneck skip → Up 1 | 4D × 7 × 7 → D × 14 × 14 |
| Concat middle skip → Up 2 | 2D × 14 × 14 → D × 28 × 28 |
| Concat first skip → output | 2D × 28 × 28 → 1 × 28 × 28 |

## 1.2 Adding noise

For each of three test digits, I reuse one noise tensor across $\sigma \in \{0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$, forming $x + \sigma\varepsilon$. Only display values are clipped. The digit becomes harder to separate from the background as $\sigma$ rises.

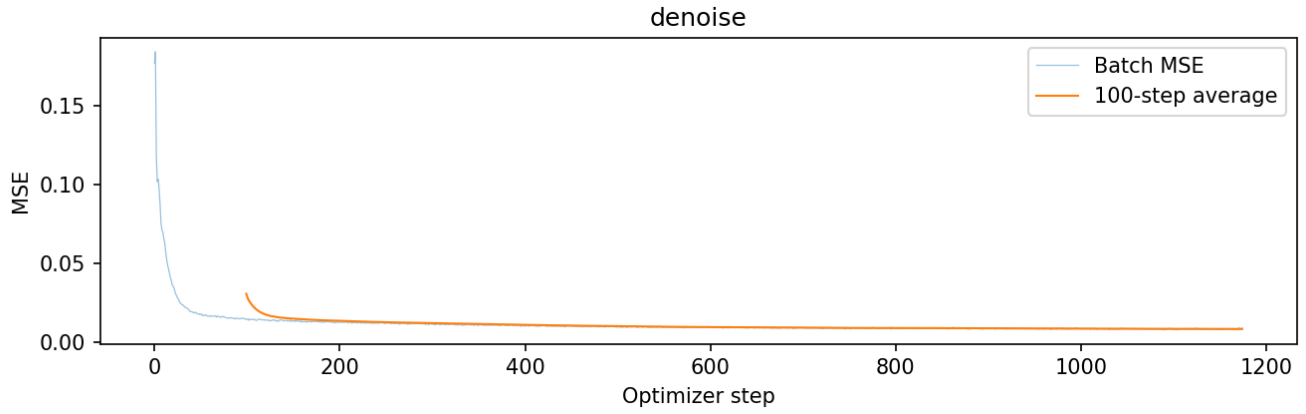



Three test digits across seven noise levels

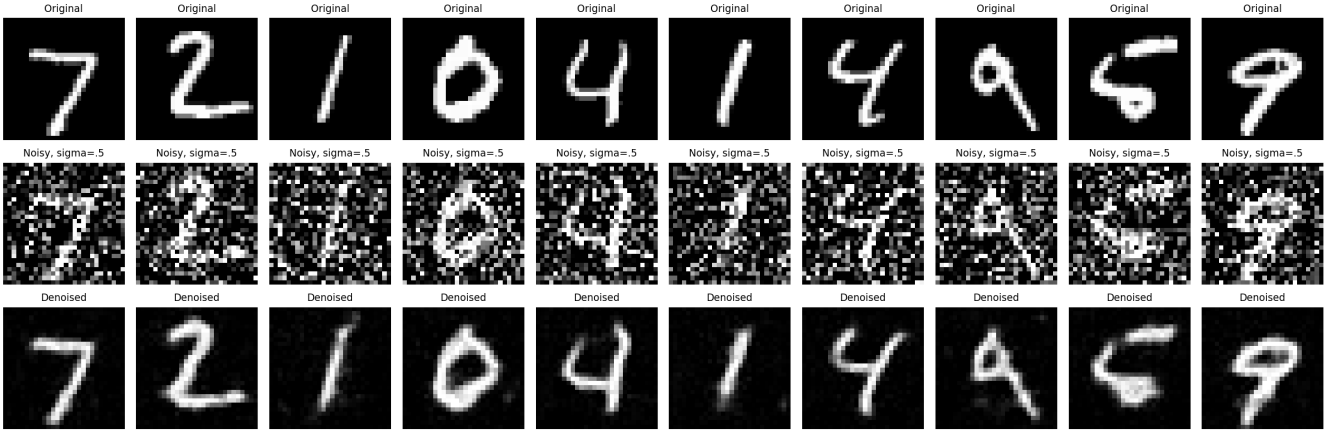
## 1.2.1 Learning a denoiser

I train a D = 128 UNet for five epochs with batch size 256 and learning rate $10^{-4}$. The objective is MSE between the predicted clean digit and the clean target, using $\sigma = 0.5$. The examples use the same test digits and noise at both checkpoints.

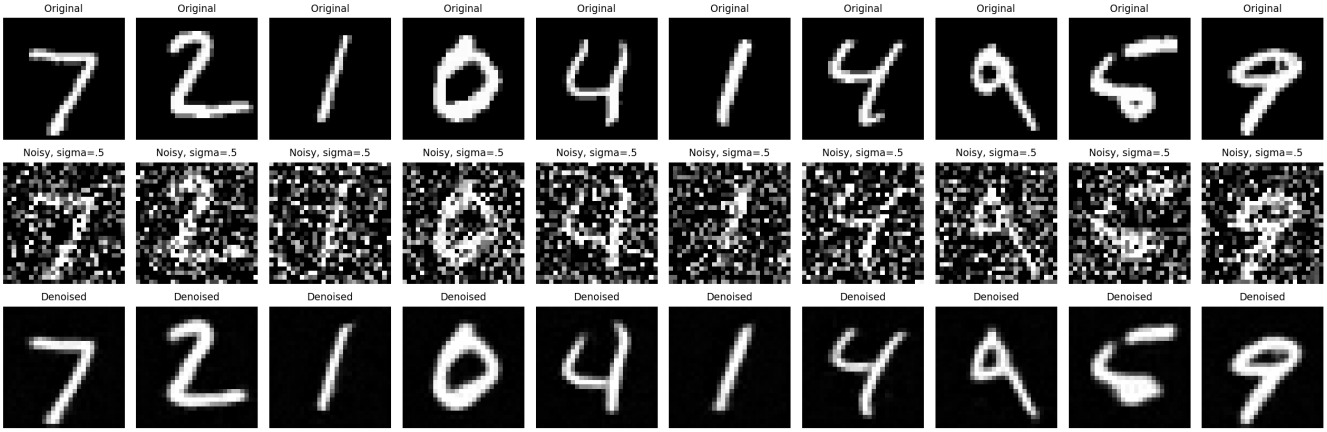
| Epoch | Mean training MSE | Learning rate |
|---|---|---|
| 1 | 0.020514 | 0.0001 |
| 5 | 0.008261 | 0.0001 |




Denoiser training loss: every batch and a 100-step moving average




Epoch 1

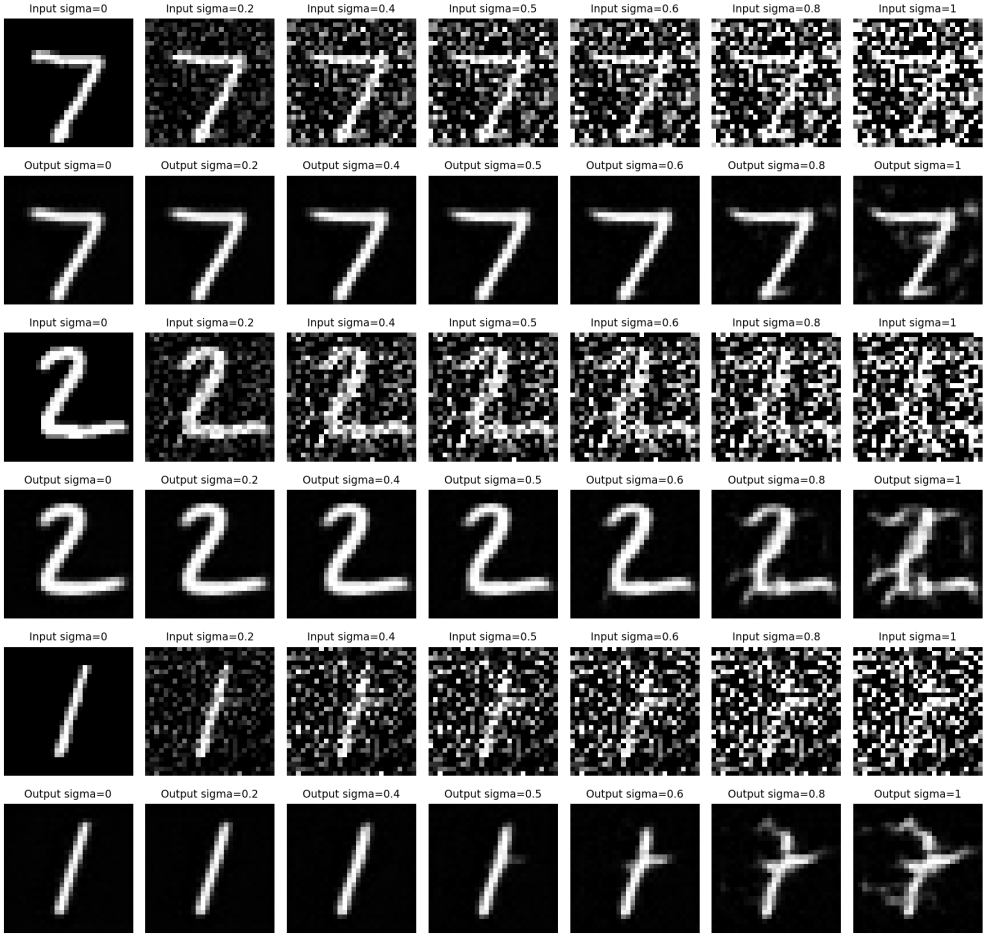



Epoch 5

Epoch 1 already recovers recognizable digits but leaves some background noise and soft edges. By epoch 5, the backgrounds are cleaner and most strokes are recovered, with some smoothing compared with the originals.

## 1.2.2 Out-of-distribution noise

The epoch-5 model was trained only at σ = 0.5. Below, each pair of rows shows inputs and outputs for one test digit, with the same noise tensor across all seven levels. Around the training level, the displayed digits remain recognizable. At σ = 0.8 and 1.0, distortions increase; the displayed 1 gains an angled stroke. At σ = 0, the network still smooths the clean input slightly because its learned mapping is not an identity function.

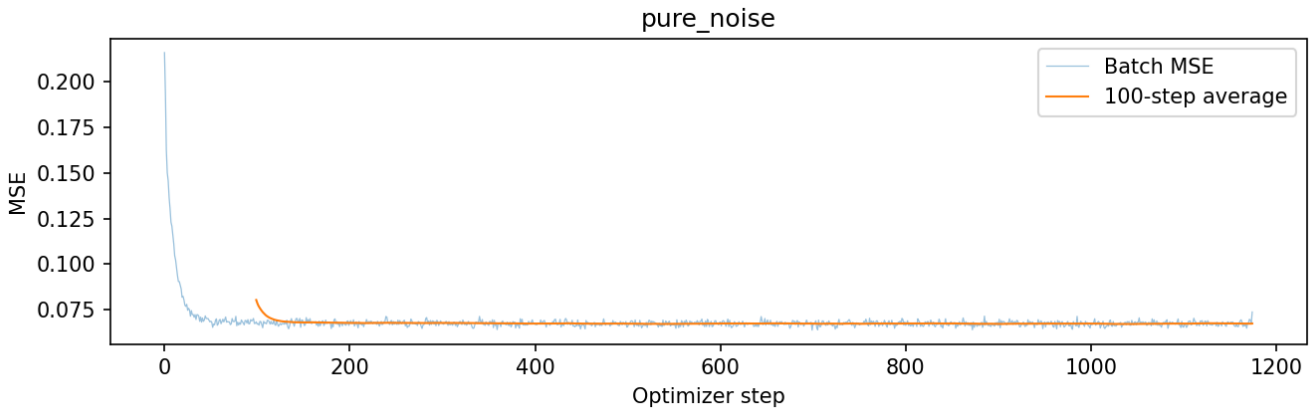



OOD comparison: seven noise levels for each of three test digits

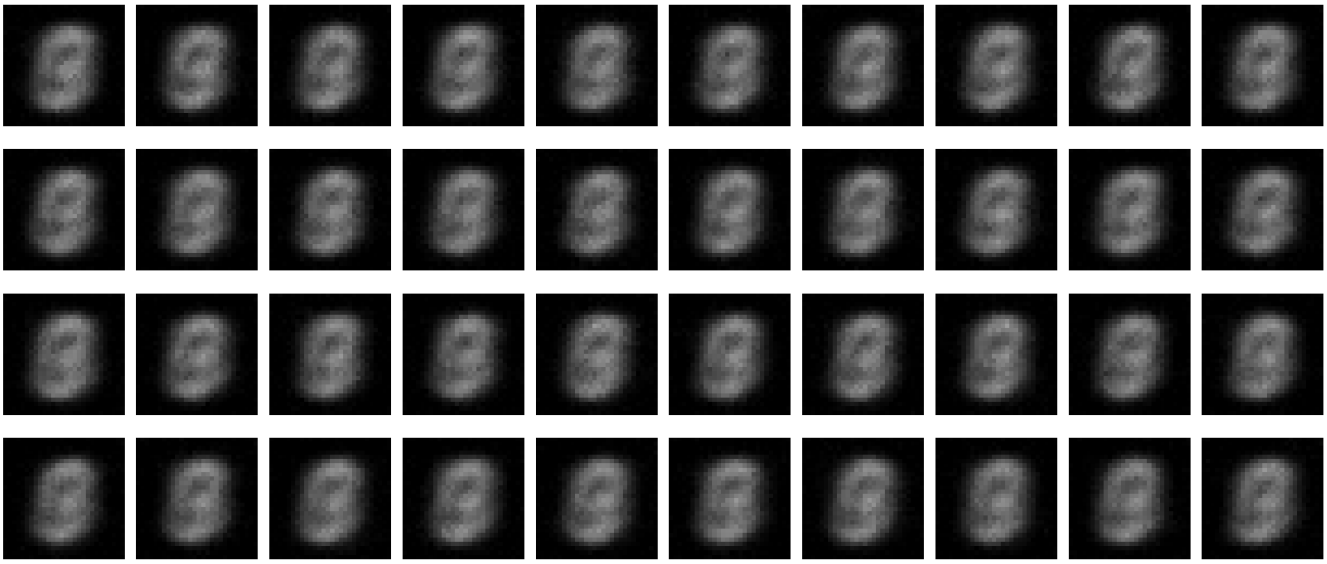
# 1.2.3 Independent pure-noise baseline

I train a fresh D = 128 UNet for five epochs with the same batch size and optimizer, but replace each input with pure Gaussian noise independent of the clean digit target. No parameters are reused from the denoiser.

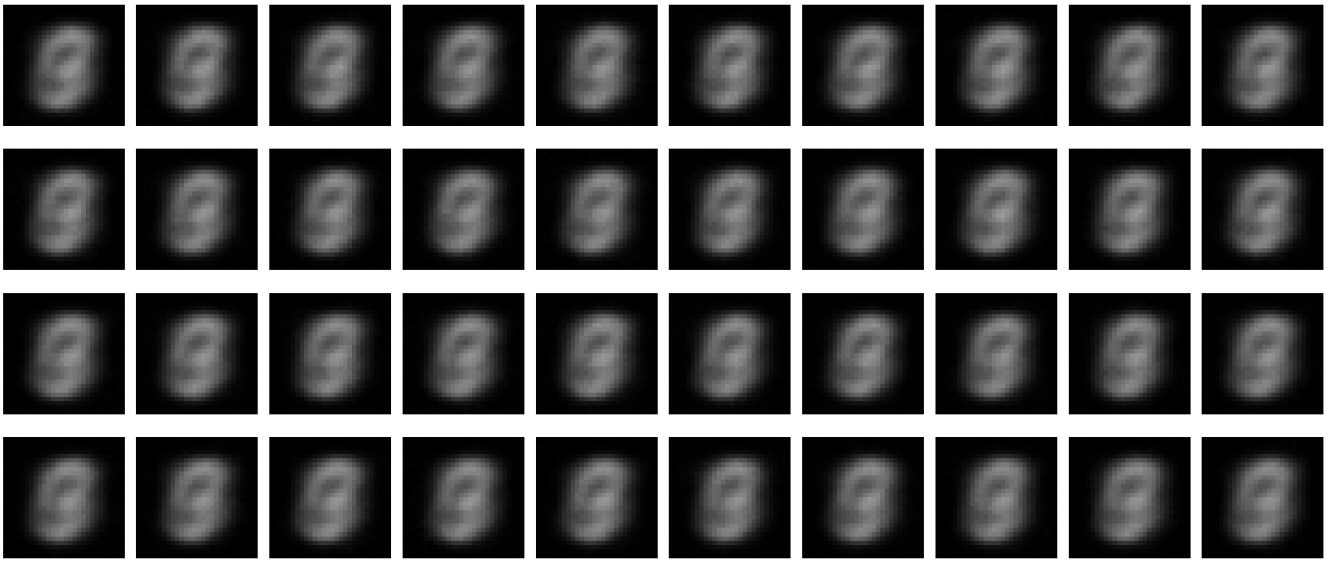
| Epoch | Mean training MSE | Learning rate |
|---|---|---|
| 1 | 0.073120 | 0.0001 |
| 5 | 0.067410 | 0.0001 |




Independent-noise baseline training loss



Epoch 1



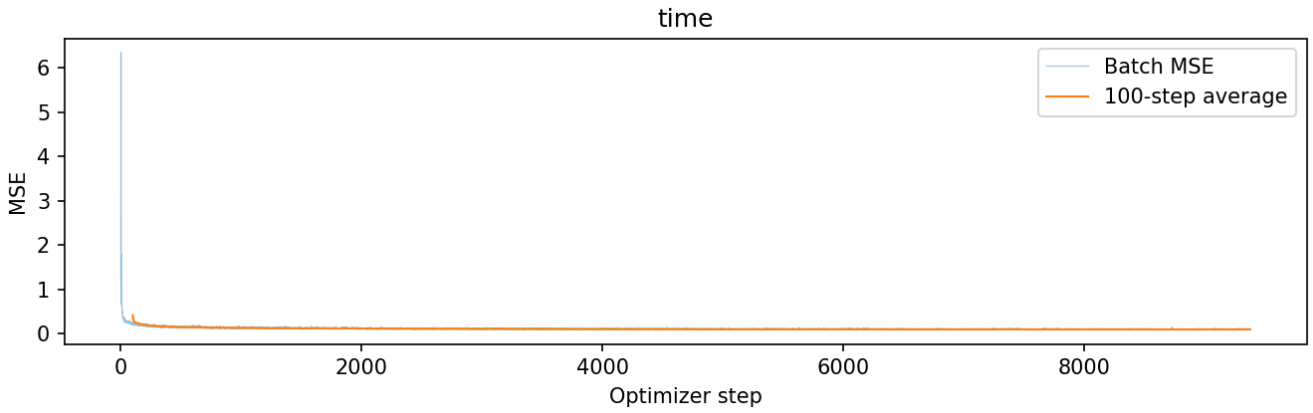
Epoch 5

The outputs converge toward nearly the same blurry oval with overlapping digit strokes, rather than distinct digits. For squared error, the optimal predictor is E[x | ε]. Since ε and x are independent, this equals E[x], the average training digit. More training cannot recover a target that the input does not identify.

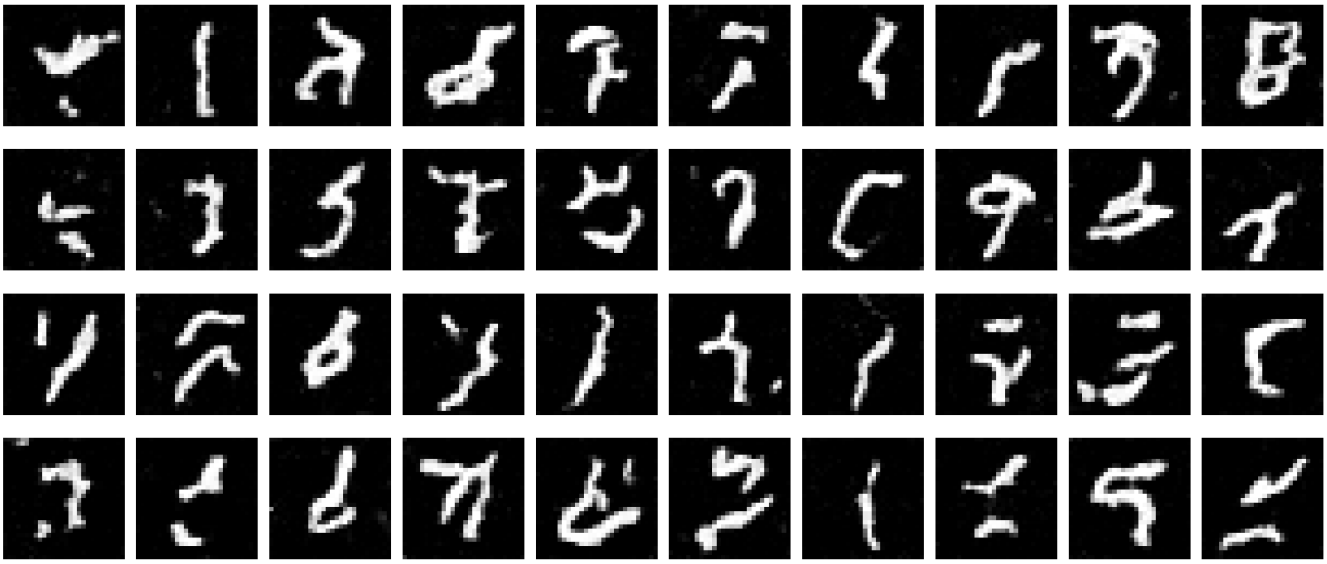
# 2.1–2.3 Time-conditioned flow matching

I use D = 64 and add two time embeddings, each Linear → GELU → Linear, at the bottleneck and first upsampled feature map. For every example I sample t uniformly in [0, 1], form $(1 - t)x_0 + tx_1$, and regress the velocity $x_1 - x_0$. Training uses batch size 64 for ten epochs, initial learning rate 0.01, and an exponential scheduler with $\gamma = 0.1^{(1/10)}$. Sampling starts from the same 40 noise tensors at each checkpoint and uses 50 Euler steps.

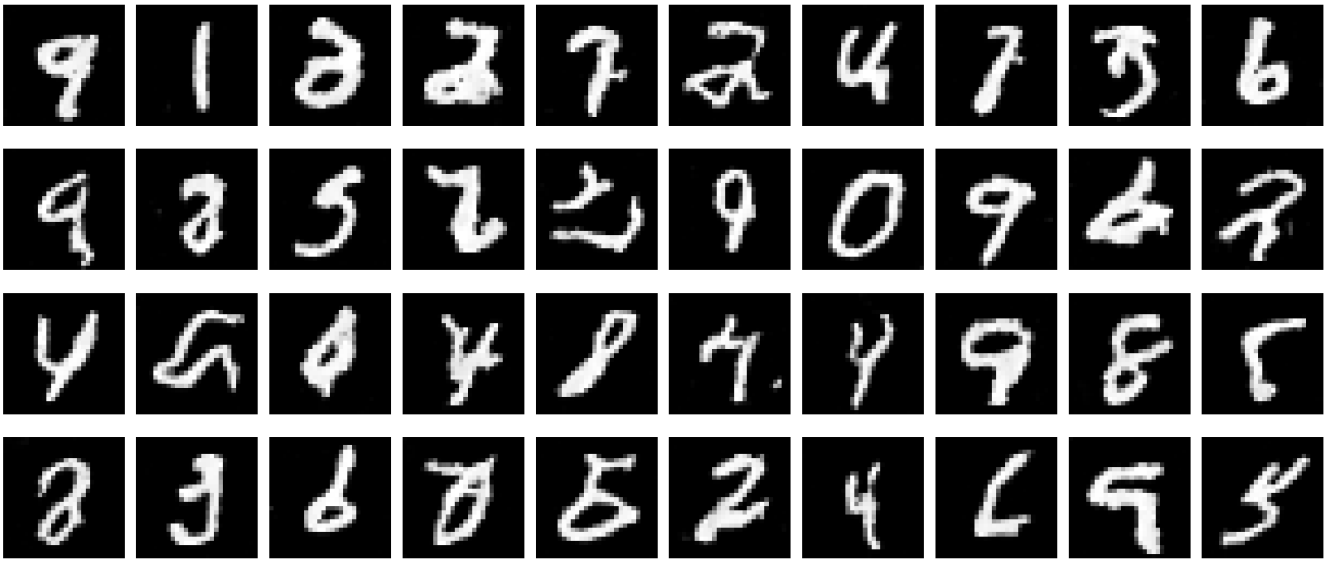
| Epoch | Mean training MSE | Learning rate |
|---|---|---|
| 1 | 0.177327 | 0.01 |
| 5 | 0.099591 | 0.00398107 |
| 10 | 0.091726 | 0.00125893 |



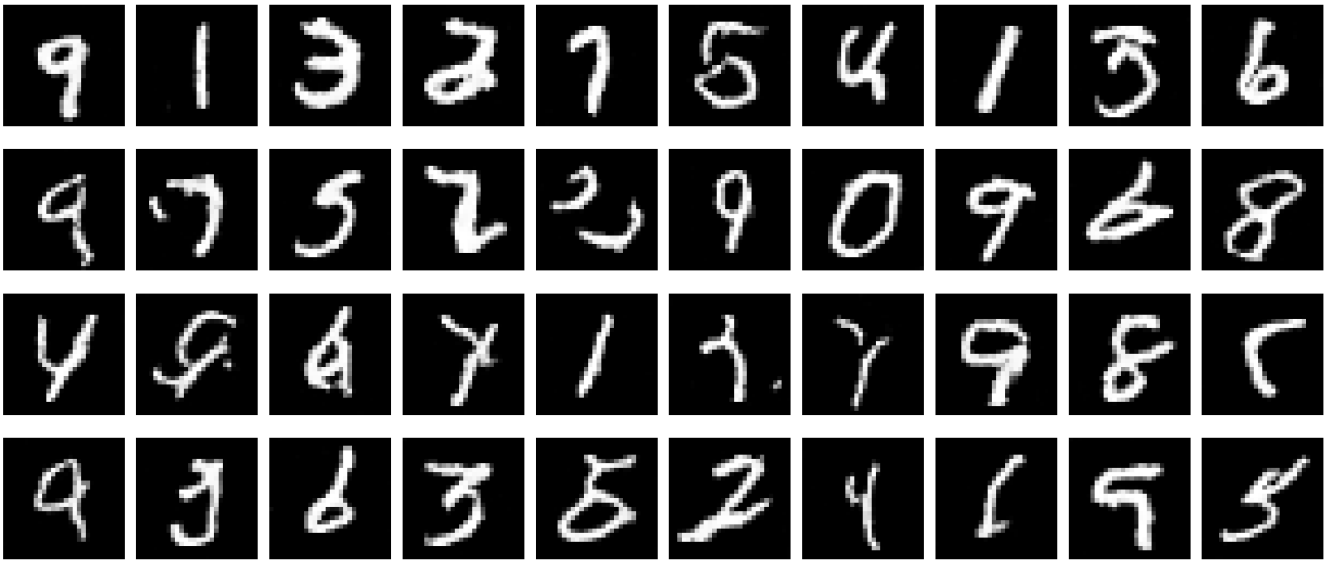

Time-conditioned velocity training loss



Epoch 1



Epoch 5



Epoch 10

At epoch 1, many outputs have broken or ambiguous strokes. At epoch 10, most are recognizable digits, although several malformed or ambiguous shapes remain. The repeated starting noise makes these changes directly comparable. Unlike independent-noise regression, the time-conditioned input follows a path toward a corresponding target, and sampling repeatedly updates the state along the learned velocity field.

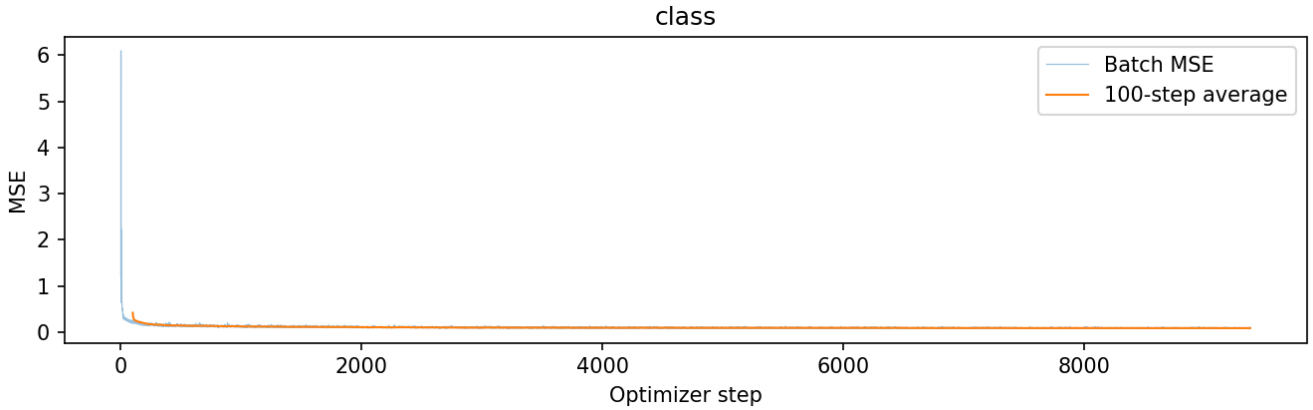
# 2.4–2.6 Class-conditioned flow matching

I embed a ten-dimensional one-hot label with two more Linear → GELU → Linear blocks. Class embeddings scale features while time embeddings shift them. With probability 0.1, the one-hot vector is replaced by all zeros during training, so the same network learns an unconditional branch. D = 64, batch size 64, ten epochs, Adam, and the learning-rate schedule match the time-only run.

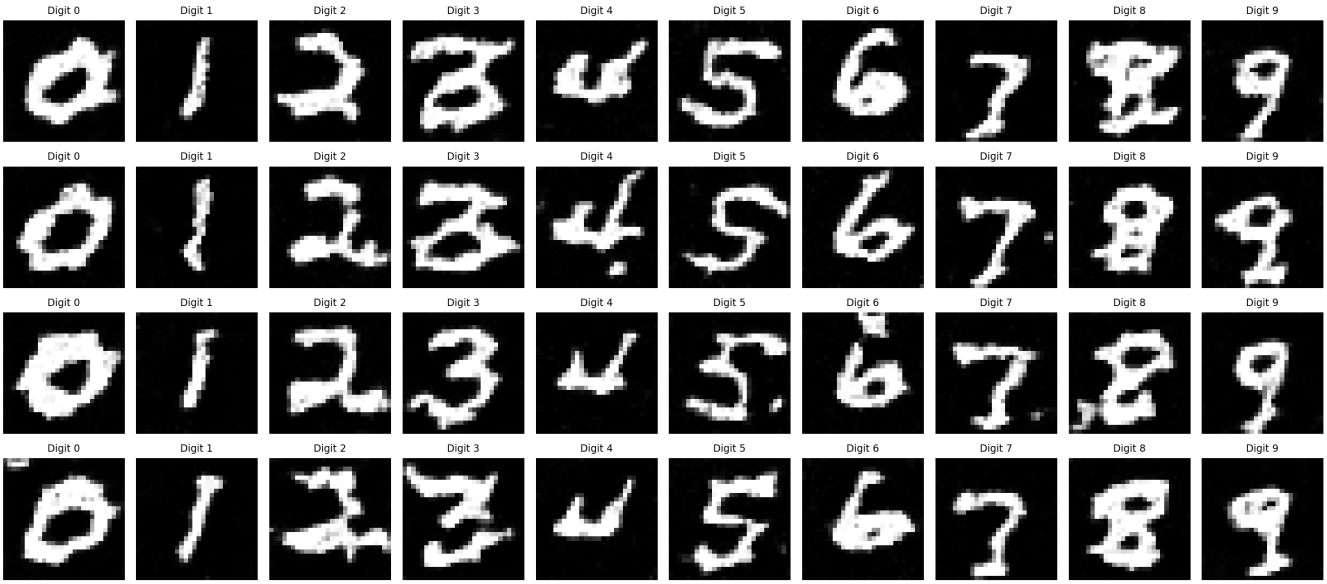
Part B guidance: $v = v_u + 5(v_c - v_u)$

Sampling uses 300 Euler steps. Each grid has four samples of each digit, with columns ordered 0–9. The same initial noise is reused across epochs and in the scheduler comparison.

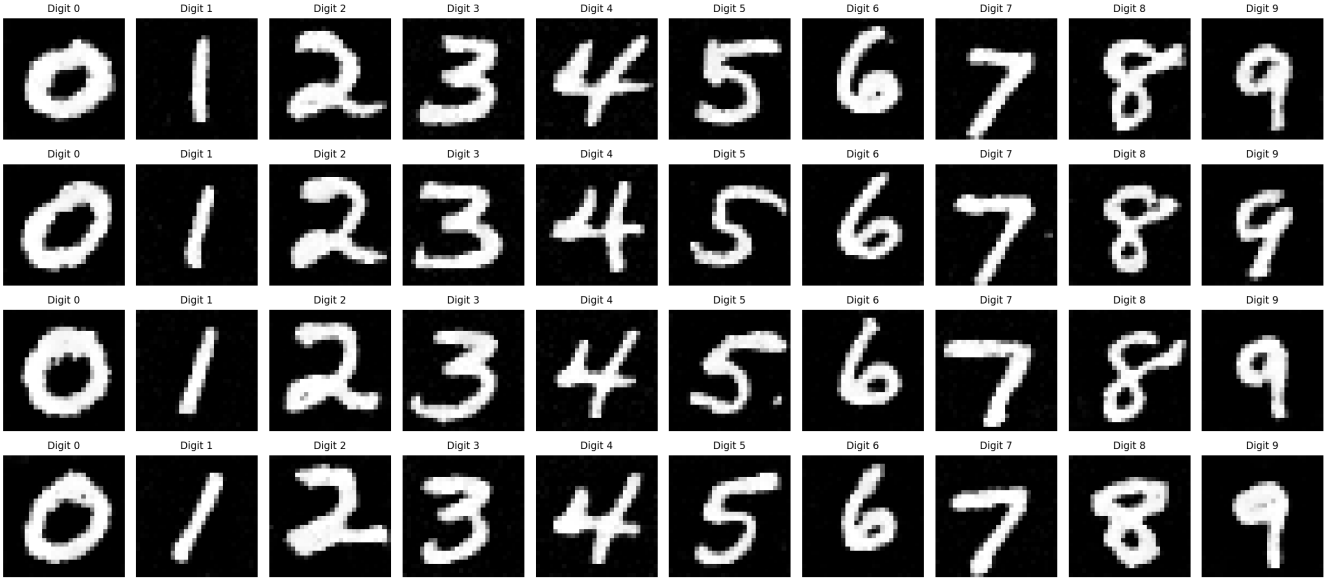
| Epoch | Mean training MSE | Learning rate |
|---|---|---|
| 1 | 0.172907 | 0.01 |
| 5 | 0.092549 | 0.00398107 |
| 10 | 0.084821 | 0.00125893 |



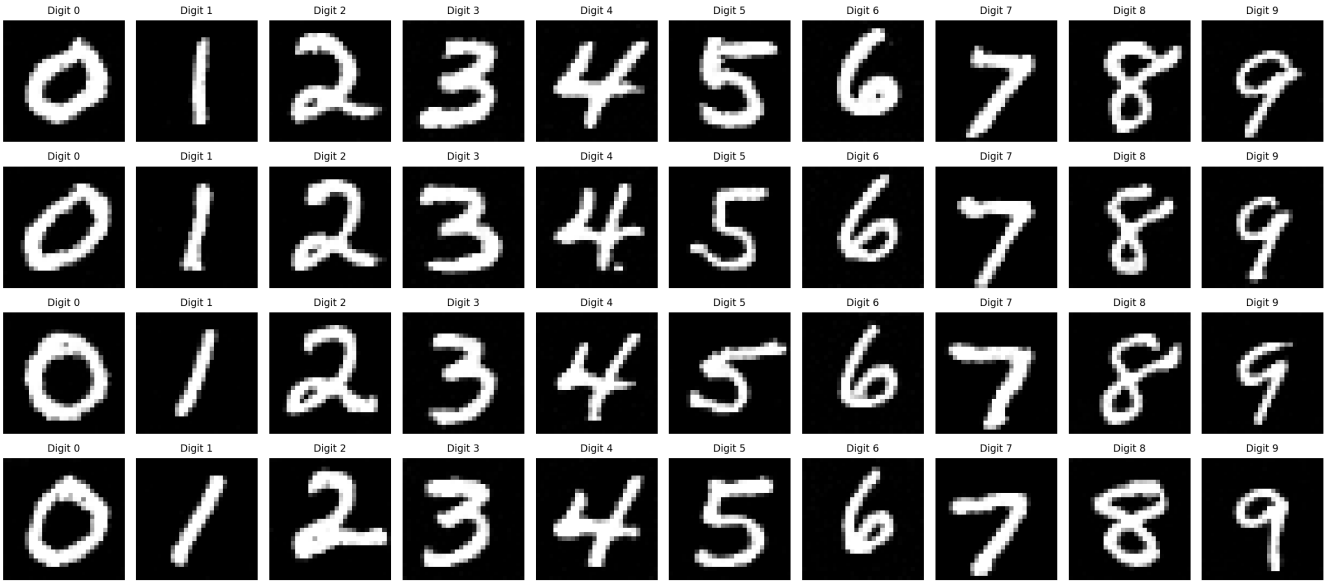

Class-conditioned velocity training loss




Epoch 1




Epoch 5



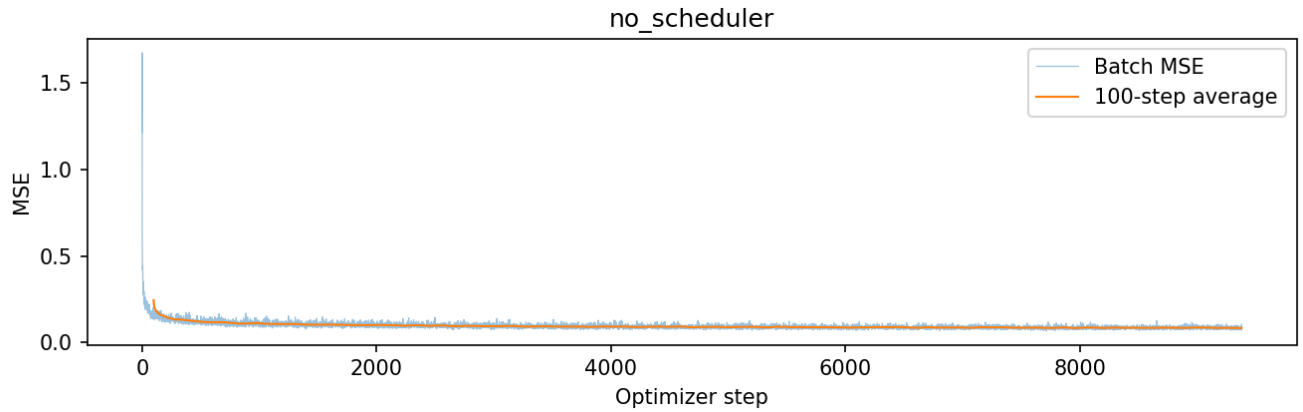

Epoch 10

By epoch 10, the displayed samples are recognizable as their requested classes, with variation in stroke width, slant, and loop shape. Compared with the time-only grid, the labels give direct control over which digit appears. Some shapes remain stylized or imperfect; this small visual grid is not a measured classification-accuracy evaluation.

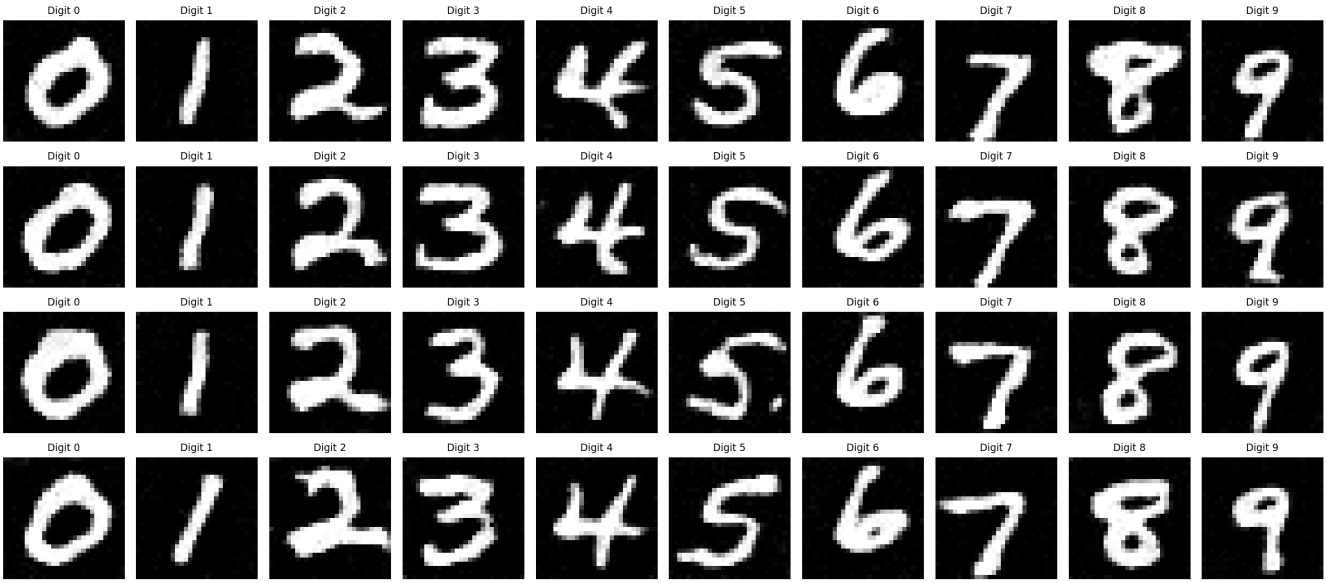
# 2.6 Removing the learning-rate scheduler

I replace the decaying schedule with a fixed learning rate of 0.001, compensating for the lack of late-stage decay by lowering the initial rate tenfold. Architecture, seed, training split, batch size, ten-epoch budget, class dropout, sampling steps, noise, and guidance are kept the same.

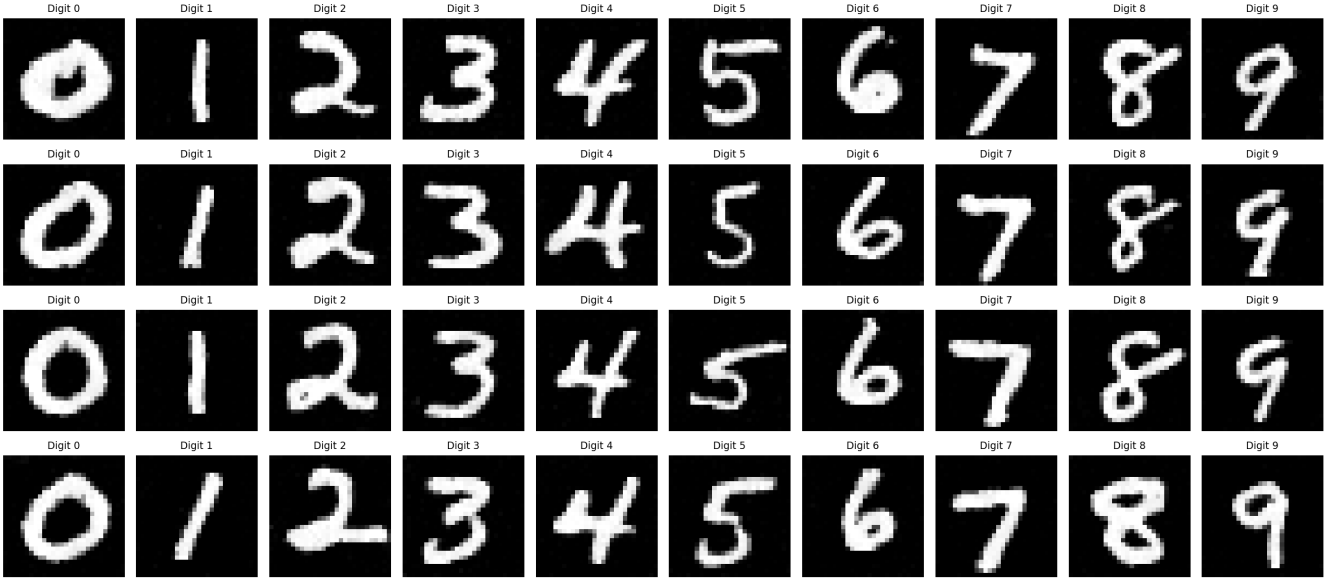
| Epoch | Mean training MSE | Learning rate |
|---|---|---|
| 1 | 0.136445 | 0.001 |
| 5 | 0.091889 | 0.001 |
| 10 | 0.085854 | 0.001 |



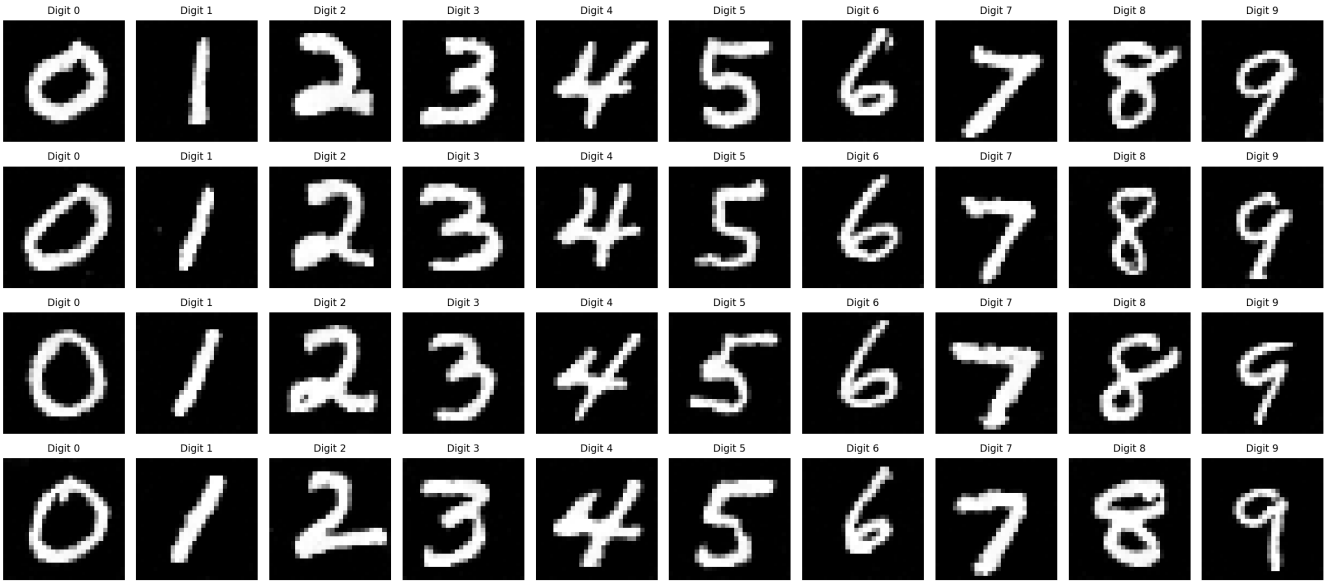

Fixed-learning-rate velocity training loss




Epoch 1




Epoch 5




Epoch 10

At epoch 10, both runs produce recognizable, class-consistent digits in the displayed 40-sample grids. The fixed-LR run has some thicker and more angular strokes. Its final mean training MSE is 0.085854, versus 0.084821 with scheduling, about 1.2% higher. Lowering the fixed rate provides a stable update size throughout training, avoiding the large late-stage updates that a constant 0.01 would retain. In this comparison the visual results are similar, but the training-loss difference and limited sample grid do not establish equal generation quality.

# Reproducibility and sources

Part A sampling and editing functions are defined directly in parta.ipynb. Part B network definitions, training, checkpointing, and sampling are defined directly in partb.ipynb. Both notebooks run without separate project Python modules. Both notebooks preserve the course starter structure and include executed plots and results. Model settings and training hyperparameters are defined in the notebooks.

Part A model revision: 9fa8836b553976b88a3b27649ed1628499e02bf9. Text encoder revision: 70d244cc86ccca08cf5af4e1e306ecf908b1ad5e. The course-supplied PixNerd adapter, support code, and starter notebooks are credited to the course staff. Implementation and report were developed with Codex assistance; the photos and two sketches are my own.

- Course assignment, Part A
- Course assignment, Part B
- Hokusai web input, CC0/public domain, via Wikimedia Commons and the Art Institute of Chicago